Tarea 5

Generado por Doxygen 1.16.1

Capítulo 1

Índice jerárquico

1.1. Jerarquía de clases

Este listado de herencia está ordenado de forma general pero no está en orden alfabético estricto:

Coord . . . . . ??
Extent . . . . . ??
Fragment . . . . . ??
Image . . . . . ??
Kernel . . . . . ??
Light . . . . . ??
Material . . . . . ??
Payload . . . . . ??
Pipeline . . . . . ??
QDockWidget
- LightsDock . . . . . ??
- MaterialDock . . . . . ??
- PostProcessingDock . . . . . ??
- TransformDock . . . . . ??
- ViewDock . . . . . ??

QMainWindow
- Window . . . . . ??

QWidget
- Canvas . . . . . ??

Rect . . . . . ??
Transform . . . . . ??
Vertex . . . . . ??

Capítulo 2

Índice de clases

2.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

Canvas
: Lienzo donde se muestra la imagen en la ventana . . . . . ??

Coord
: Coordenadas de un pixel en una imagen . . . . . ??

Extent
: Tamaño de una región rectangular en una imagen . . . . . ??

Fragment
: Fragmento generado por el rasterizador . . . . . ??

Image
: Imagen con información del tamaño . . . . . ??

Kernel
: Kernel simétrico para operaciones de convolución . . . . . ??

Light
: Propiedades de una luz puntual . . . . . ??

LightsDock
: Puerto para el control de las luces . . . . . ??

Material
: Propiedades de un material . . . . . ??

MaterialDock
: Puerto para el control del material . . . . . ??

Payload
: Carga que se transfiere entre etapas de la línea de renderizado . . . . . ??

Pipeline
: Línea de renderizado por rasterización . . . . . ??

PostProcessingDock . . . . . ??

Rect
: Región rectangular de pixeles en una imagen . . . . . ??

Transform
: Transformación de un objeto . . . . . ??

TransformDock
: Puerto para el control de la transformación . . . . . ??

Vertex
: Vértice que se procesa en la línea de renderizado . . . . . ??

ViewDock
: Puerto para el control de la proyección . . . . . ??

Window
: Ventana principal de la aplicación . . . . . ??

Capítulo 3

Índice de archivos

3.1. Lista de archivos

Lista de todos los archivos documentados y con breves descripciones:

include/app.hpp . . . . . ??
include/dip.hpp . . . . . ??
include/gmath.hpp . . . . . ??
include/gui.hpp . . . . . ??
include/pipeline.hpp . . . . . ??

Capítulo 4

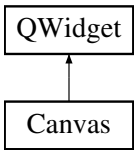
Documentación de clases

4.1. Referencia de la estructura Canvas

Lienzo donde se muestra la imagen en la ventana.

`#include <gui.hpp>`

Diagrama de herencia de Canvas



Métodos públicos

- void paintEvent (QPaintEvent ∗event)
  Respuesta a un evento de redibujar el lienzo.

4.1.1. Descripción detallada

Lienzo donde se muestra la imagen en la ventana.

4.1.2. Documentación de funciones miembro

4.1.2.1. paintEvent()

```
void Canvas::paintEvent (
            QPaintEvent * event)
```

Respuesta a un evento de redibujar el lienzo.

Parámetros

event	el evento recibido

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.2. Referencia de la estructura Coord

Coordenadas de un pixel en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **x**
- uint64_t **y**

4.2.1. Descripción detallada

Coordenadas de un pixel en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.3. Referencia de la estructura Extent

Tamaño de una región rectangular en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **width**
- uint64_t **height**

4.3.1. Descripción detallada

Tamaño de una región rectangular en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.4. Referencia de la estructura Fragment

Fragmento generado por el rasterizador.

```
#include <pipeline.hpp>
```

Atributos públicos

- uint64_t **x**
- uint64_t **y**
- double **u**
- double **v**
- double **depth**
- double **color** [4]

4.4.1. Descripción detallada

Fragmento generado por el rasterizador.

La documentación de esta estructura está generada del siguiente archivo:

- include/pipeline.hpp

4.5. Referencia de la estructura Image

Imagen con información del tamaño.

```
#include <dip.hpp>
```

Atributos públicos

- Extent **extent**
- void * **data**

4.5.1. Descripción detallada

Imagen con información del tamaño.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.6. Referencia de la estructura Kernel

Kernel simétrico para operaciones de convolución.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **radius**
- void * **data**

4.6.1. Descripción detallada

Kernel simétrico para operaciones de convolución.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.7. Referencia de la estructura Light

Propiedades de una luz puntual.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **ambient** [3]
- double **diffuse** [3]
- double **specular** [3]

4.7.1. Descripción detallada

Propiedades de una luz puntual.

La documentación de esta estructura está generada del siguiente archivo:

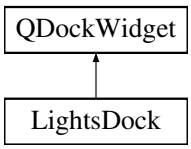
- include/app.hpp

4.8. Referencia de la estructura LightsDock

Puerto para el control de las luces.

```
#include <gui.hpp>
```

Diagrama de herencia de LightsDock



Métodos públicos

- **LightsDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QGridLayout **layout**
- QComboBox **dropdown**
- QCheckBox **checkbox**
- QLabel **row_labels** [3]
- QLabel **col_labels** [3]
- QDoubleSpinBox **amb_boxes** [3]
- QDoubleSpinBox **diff_boxes** [3]
- QDoubleSpinBox **spec_boxes** [3]

4.8.1. Descripción detallada

Puerto para el control de las luces.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.9. Referencia de la estructura Material

Propiedades de un material.

```
#include <app.hpp>
```

Atributos públicos

- double **ambient** [3]
- double **diffuse** [3]
- double **specular** [3]
- double **shininess**

4.9.1. Descripción detallada

Propiedades de un material.

La documentación de esta estructura está generada del siguiente archivo:

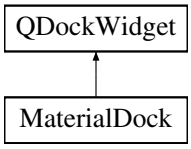
- include/app.hpp

4.10. Referencia de la estructura MaterialDock

Puerto para el control del material.

```
#include <gui.hpp>
```

Diagrama de herencia de MaterialDock



Métodos públicos

- **MaterialDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QGridLayout **layout**
- QLabel **row_labels** [3]
- QLabel **col_labels** [3]
- QDoubleSpinBox **amb_boxes** [3]
- QDoubleSpinBox **diff_boxes** [3]
- QDoubleSpinBox **spec_boxes** [3]
- QLabel **sh_label**
- QDoubleSpinBox **sh_box**

4.10.1. Descripción detallada

Puerto para el control del material.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.11. Referencia de la estructura Payload

Carga que se transfiere entre etapas de la línea de renderizado.

```
#include <app.hpp>
```

Atributos públicos

- double **view** [3]
- double **normal** [3]
- double **light** [NUM_LIGHTS][3]
- double **color** [3]

4.11.1. Descripción detallada

Carga que se transfiere entre etapas de la línea de renderizado.

La documentación de esta estructura está generada del siguiente archivo:

- include/app.hpp

4.12. Referencia de la estructura Pipeline

Línea de renderizado por rasterización.

```
#include <pipeline.hpp>
```

Métodos públicos

- void **clear_buffers** ()

  Limpia los buffers de color y profundidad.
- void **draw** ()

  Dibuja triángulos.
- bool clipping_test ()

  Realiza la prueba de recorte.
- void **to_plane** ()

  Pasa la primitiva actual al plano de proyección.
- void **to_viewport** ()

  Pasa la primitiva actual a la región de dibujo de de la imagen.
- void **rasterize** ()

  Rasteriza la primitiva actual.
- bool depth_test ()

  Realiza la prueba de profundidad.
- void **build_payload** ()

  Construye la carga del fragmento actual.
- void **blend_fragment** ()

  Asigna el color del fragmento al buffer de color.

Atributos públicos

- uint64_t **num_primitives**
- uint64_t **width**
- uint64_t **height**
- uint64_t **primitive_id**
- uint64_t **vertex_id**
- bool **front_face_clockwise**
- Fragment **fragment**
- Payload **out** [3]
- Payload **in**
- uint64_t **viewport** [2][2]
- uint64_t **raster_rect** [2][2]
- double **positions_3d** [3][3]
- double **boundary_3d** [2][3]
- double **positions_2d** [3][2]
- double **boundary_2d** [2][2]
- void ∗ **color_buffer**
- void ∗ **depth_buffer**
- void ∗ **vertex_buffer**
- Payload(∗ **vertex_shader**)(Vertex, double ∗)
- void(∗ **fragment_shader**)(Payload, double ∗)

4.12.1. Descripción detallada

Línea de renderizado por rasterización.

4.12.2. Documentación de funciones miembro

4.12.2.1. clipping_test()

```
bool Pipeline::clipping_test ()
```

Realiza la prueba de recorte.

Devuelve

`true` si y solo si la primitiva actual está en el volumen de visión

4.12.2.2. depth_test()

```
bool Pipeline::depth_test ()
```

Realiza la prueba de profundidad.

Devuelve

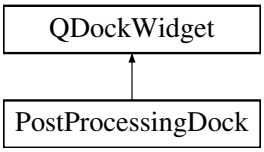
`true` si y solo si el fragmento actual es visible dada su profundidad

La documentación de esta estructura está generada del siguiente archivo:

- include/pipeline.hpp

4.13. Referencia de la estructura PostProcessingDock

Diagrama de herencia de PostProcessingDock



Métodos públicos

- void **syncValues** ()

Atributos públicos

- QWidget **widget**
- QGridLayout **layout**
- QCheckBox **checkbox**
- QLabel **label**
- QDoubleSpinBox **boxes** [3][3]

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.14. Referencia de la estructura Rect

Región rectangular de pixeles en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- Coord **start**
- Coord **end**

4.14.1. Descripción detallada

Región rectangular de pixeles en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.15. Referencia de la estructura Transform

Transformación de un objeto.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **rotation** [4]
- double **scale** [3]

4.15.1. Descripción detallada

Transformación de un objeto.

La documentación de esta estructura está generada del siguiente archivo:

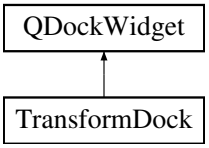
- include/app.hpp

4.16. Referencia de la estructura TransformDock

Puerto para el control de la transformación.

```
#include <gui.hpp>
```

Diagrama de herencia de TransformDock



Métodos públicos

- **TransformDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QGridLayout **layout**
- QLabel **row_labels** [3]
- QLabel **col_labels** [3]
- QDoubleSpinBox **pos_boxes** [3]
- QDoubleSpinBox **rot_boxes** [3]
- QDoubleSpinBox **scale_boxes** [3]

4.16.1. Descripción detallada

Puerto para el control de la transformación.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.17. Referencia de la estructura Vertex

Vértice que se procesa en la línea de renderizado.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **normal** [3]
- double **color** [3]

4.17.1. Descripción detallada

Vértice que se procesa en la línea de renderizado.

La documentación de esta estructura está generada del siguiente archivo:

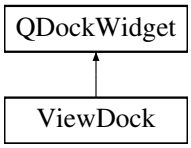
- include/app.hpp

4.18. Referencia de la estructura ViewDock

Puerto para el control de la proyección.

```
#include <gui.hpp>
```

Diagrama de herencia de ViewDock



Métodos públicos

- **ViewDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QVBoxLayout **layout**
- QGroupBox **cam_group**
- QVBoxLayout **cam_group_layout**
- QRadioButton **cam_group_buttons** [2]
- QGroupBox **proj_group**
- QVBoxLayout **proj_group_layout**
- QRadioButton **proj_group_buttons** [2]

4.18.1. Descripción detallada

Puerto para el control de la proyección.

La documentación de esta estructura está generada del siguiente archivo:

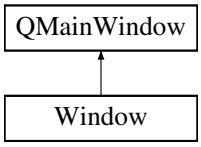
- include/gui.hpp

4.19. Referencia de la estructura Window

Ventana principal de la aplicación.

```
#include <gui.hpp>
```

Diagrama de herencia de Window



Métodos públicos

- **Window** ()

  Construye una nueva ventana.
- void **draw** ()

  Renderiza las escenas vistas por las cámaras y actualiza el lienzo.
- void **sync_all** ()
- void **connect_all** ()

  Conecta las señales de los puertos.
- void slot_pos_x (double value)

  Respuesta a un cambio en la posición x en el puerto de transformación.
- void slot_pos_y (double value)

  Respuesta a un cambio en la posición y en el puerto de transformación.
- void slot_pos_z (double value)

  Respuesta a un cambio en la posición z en el puerto de transformación.
- void slot_rot_x (double value)

Respuesta a un cambio en la rotación x en el puerto de transformación.

- void slot_rot_y (double value)

  Respuesta a un cambio en la rotación y en el puerto de transformación.
- void slot_rot_z (double value)

  Respuesta a un cambio en la rotación z en el puerto de transformación.
- void slot_scale_x (double value)

  Respuesta a un cambio en la escala x en el puerto de transformación.
- void slot_scale_y (double value)

  Respuesta a un cambio en la escala y en el puerto de transformación.
- void slot_scale_z (double value)

  Respuesta a un cambio en la escala z en el puerto de transformación.
- void slot_proj (bool checked)

  Respuesta a un cambio de la proyección en el puerto de vista.
- void slot_cam (bool checked)

  Respuesta a un cambio de la cámara en el puerto de vista.
- void slot_light_id (int index)

  Resouesta a un cambio de la luz seleccionada en el puerto de luces.
- void slot_light_state (Qt::CheckState state)

  Respuesta a un cambio del estado encendido en el puerto de luces.
- void slot_light_amb_r (double value)

  Respuesta a un cambio en la componente de reflexión ambiental roja en el puerto de luces.
- void slot_light_amb_g (double value)

  Respuesta a un cambio en la componente de reflexión ambiental verde en el puerto de luces.
- void slot_light_amb_b (double value)

  Respuesta a un cambio en la componente de reflexión ambiental azul en el puerto de luces.
- void slot_light_diff_r (double value)

  Respuesta a un cambio en la componente de reflexión difusa roja en el puerto de luces.
- void slot_light_diff_g (double value)

  Respuesta a un cambio en la componente de reflexión difusa verde en el puerto de luces.
- void slot_light_diff_b (double value)

  Respuesta a un cambio en la componente de reflexión difusa azul en el puerto de luces.
- void slot_light_spec_r (double value)

  Respuesta a un cambio en la componente de reflexión especular roja en el puerto de luces.
- void slot_light_spec_g (double value)

  Respuesta a un cambio en la componente de reflexión especular verde en el puerto de luces.
- void slot_light_spec_b (double value)

  Respuesta a un cambio en la componente de reflexión especular azul en el puerto de luces.
- void slot_material_amb_r (double value)

  Respuesta a un cambio en el coeficiente de reflexión ambiental rojo en el puerto de material.
- void slot_material_amb_g (double value)

  Respuesta a un cambio en el coeficiente de reflexión ambiental verde en el puerto de material.
- void slot_material_amb_b (double value)

  Respuesta a un cambio en el coeficiente de reflexión ambiental azul en el puerto de material.
- void slot_material_diff_r (double value)

  Respuesta a un cambio en el coeficiente de reflexión difusa rojo en el puerto de material.
- void slot_material_diff_g (double value)

  Respuesta a un cambio en el coeficiente de reflexión difusa verde en el puerto de material.
- void slot_material_diff_b (double value)

  Respuesta a un cambio en el coeficiente de reflexión difusa azul en el puerto de material.
- void slot_material_spec_r (double value)

  Respuesta a un cambio en el coeficiente de reflexión especular rojo en el puerto de material.

- void slot_material_spec_g (double value)

  Respuesta a un cambio en el coeficiente de reflexión especular verde en el puerto de material.
- void slot_material_spec_b (double value)

  Respuesta a un cambio en el coeficiente de reflexión especular azul rojo en el puerto de material.
- void slot_material_sh (double value)

  Respuesta a un cambio en el brillo especular en el puerto de material.
- void slot_post_processing_filter (Qt::CheckState state)

  Respuesta a un cambio en el estado de aplicar filtro en el puerto de posprocesamiento.
- void slot_post_processing_kernel_value (double value)

  Respuesta a un cambio en el valor del kernel en el puerto de posprocesamiento.

Atributos públicos

- Canvas **canvas**
- ViewDock **view_dock**
- TransformDock **transform_dock**
- LightsDock **lights_dock**
- MaterialDock **material_dock**
- PostProcessingDock **post_processing_dock**

4.19.1. Descripción detallada

Ventana principal de la aplicación.

4.19.2. Documentación de funciones miembro

4.19.2.1. slot_cam()

```
void Window::slot_cam (
            bool checked)
```

Respuesta a un cambio de la cámara en el puerto de vista.

Parámetros

checked	el nuevo valor de la opción de cámara 1

4.19.2.2. slot_light_amb_b()

```
void Window::slot_light_amb_b (
            double value)
```

Respuesta a un cambio en la componente de reflexión ambiental azul en el puerto de luces.

Parámetros

value	el nuevo valor azul

4.19.2.3. slot_light_amb_g()

```
void Window::slot_light_amb_g (
            double value)
```

Respuesta a un cambio en la componente de reflexión ambiental verde en el puerto de luces.

Parámetros

value	el nuevo valor verde

4.19.2.4. slot_light_amb_r()

```
void Window::slot_light_amb_r (
            double value)
```

Respuesta a un cambio en la componente de reflexión ambiental roja en el puerto de luces.

Parámetros

value	el nuevo valor rojo

4.19.2.5. slot_light_diff_b()

```
void Window::slot_light_diff_b (
            double value)
```

Respuesta a un cambio en la componente de reflexión difusa azul en el puerto de luces.

Parámetros

value	el nuevo valor azul

4.19.2.6. slot_light_diff_g()

```
void Window::slot_light_diff_g (
            double value)
```

Respuesta a un cambio en la componente de reflexión difusa verde en el puerto de luces.

Parámetros

value	el nuevo valor verde

4.19.2.7. slot_light_diff_r()

```
void Window::slot_light_diff_r (
            double value)
```

Respuesta a un cambio en la componente de reflexión difusa roja en el puerto de luces.

Parámetros

value	el nuevo valor rojo

4.19.2.8. slot_light_id()

```
void Window::slot_light_id (
            int index)
```

Resouesta a un cambio de la luz seleccionada en el puerto de luces.

Parámetros

index	el nuevo índice de la luz seleccionada

4.19.2.9. slot_light_spec_b()

```
void Window::slot_light_spec_b (
            double value)
```

Respuesta a un cambio en la componente de reflexión especular azul en el puerto de luces.

Parámetros

value	el nuevo valor azul

4.19.2.10. slot_light_spec_g()

```
void Window::slot_light_spec_g (
            double value)
```

Respuesta a un cambio en la componente de reflexión especular verde en el puerto de luces.

Parámetros

value	el nuevo valor verde

4.19.2.11. slot_light_spec_r()

```
void Window::slot_light_spec_r (
            double value)
```

Respuesta a un cambio en la componente de reflexión especular roja en el puerto de luces.

Parámetros

value	el nuevo valor rojo

4.19.2.12. slot_light_state()

```
void Window::slot_light_state (
            Qt::CheckState state)
```

Respuesta a un cambio del estado encendido en el puerto de luces.

Parámetros

state	el nuevo valor del estado encendido

4.19.2.13. slot_material_amb_b()

```
void Window::slot_material_amb_b (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión ambiental azul en el puerto de material.

Parámetros

value	el nuevo valor azul

4.19.2.14. slot_material_amb_g()

```
void Window::slot_material_amb_g (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión ambiental verde en el puerto de material.

Parámetros

value	el nuevo valor verde

4.19.2.15. slot_material_amb_r()

```
void Window::slot_material_amb_r (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión ambiental rojo en el puerto de material.

Parámetros

value	el nuevo valor rojo

4.19.2.16. slot_material_diff_b()

```
void Window::slot_material_diff_b (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión difusa azul en el puerto de material.

Parámetros

value	el nuevo valor azul

4.19.2.17. slot_material_diff_g()

```
void Window::slot_material_diff_g (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión difusa verde en el puerto de material.

Parámetros

value	el nuevo valor verde

4.19.2.18. slot_material_diff_r()

```
void Window::slot_material_diff_r (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión difusa rojo en el puerto de material.

Parámetros

value	el nuevo valor rojo

4.19.2.19. slot_material_sh()

```
void Window::slot_material_sh (
            double value)
```

Respuesta a un cambio en el brillo especular en el puerto de material.

Parámetros

value	el nuevo valor del brillo

4.19.2.20. slot_material_spec_b()

```
void Window::slot_material_spec_b (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión especular azul rojo en el puerto de material.

Parámetros

value	el nuevo valor azul

4.19.2.21. slot_material_spec_g()

```
void Window::slot_material_spec_g (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión especular verde en el puerto de material.

Parámetros

value	el nuevo valor verde

4.19.2.22. slot_material_spec_r()

```
void Window::slot_material_spec_r (
            double value)
```

Respuesta a un cambio en el coeficiente de reflexión especular rojo en el puerto de material.

Parámetros

value	el nuevo valor rojo

4.19.2.23. slot_pos_x()

```
void Window::slot_pos_x (
            double value)
```

Respuesta a un cambio en la posición x en el puerto de transformación.

Parámetros

value	el nuevo valor x

4.19.2.24. slot_pos_y()

```
void Window::slot_pos_y (
            double value)
```

Respuesta a un cambio en la posición y en el puerto de transformación.

Parámetros

value	el nuevo valor y

4.19.2.25. slot_pos_z()

```
void Window::slot_pos_z (
            double value)
```

Respuesta a un cambio en la posición z en el puerto de transformación.

Parámetros

value	el nuevo valor z

4.19.2.26. slot_post_processing_filter()

```
void Window::slot_post_processing_filter (
            Qt::CheckState state)
```

Respuesta a un cambio en el estado de aplicar filtro en el puerto de posprocesamiento.

Parámetros

checked	el nuevo valor del estado de aplicar filtro

4.19.2.27. slot_post_processing_kernel_value()

```
void Window::slot_post_processing_kernel_value (
            double value)
```

Respuesta a un cambio en el valor del kernel en el puerto de posprocesamiento.

Parámetros

value	el nuevo valor modificado

4.19.2.28. slot_proj()

```
void Window::slot_proj (
            bool checked)
```

Respuesta a un cambio de la proyección en el puerto de vista.

Parámetros

checked	el nuevo valor de la opción de proyección en perspectiva

4.19.2.29. slot_rot_x()

```
void Window::slot_rot_x (
            double value)
```

Respuesta a un cambio en la rotación x en el puerto de transformación.

Parámetros

value	el nuevo valor x

4.19.2.30. slot_rot_y()

```
void Window::slot_rot_y (
            double value)
```

Respuesta a un cambio en la rotación y en el puerto de transformación.

Parámetros

value	el nuevo valor y

4.19.2.31. slot_rot_z()

```
void Window::slot_rot_z (
            double value)
```

Respuesta a un cambio en la rotación z en el puerto de transformación.

Parámetros

value	el nuevo valor z

4.19.2.32. slot_scale_x()

```
void Window::slot_scale_x (
            double value)
```

Respuesta a un cambio en la escala x en el puerto de transformación.

Parámetros

value	el nuevo valor x

4.19.2.33. slot_scale_y()

```
void Window::slot_scale_y (
            double value)
```

Respuesta a un cambio en la escala y en el puerto de transformación.

Parámetros

value	el nuevo valor y

4.19.2.34. slot_scale_z()

```
void Window::slot_scale_z (
            double value)
```

Respuesta a un cambio en la escala z en el puerto de transformación.

Parámetros

value	el nuevo valor z

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

Capítulo 5

Documentación de archivos

5.1. Referencia del archivo include/app.hpp

```
#include <QtWidgets>
#include "tiny_obj_loader.h"
#include "gmath.hpp"
#include "dip.hpp"
```

Clases

- struct Vertex

  Vértice que se procesa en la línea de renderizado.
- struct Payload

  Carga que se transfiere entre etapas de la línea de renderizado.
- struct Light

  Propiedades de una luz puntual.
- struct Material

  Propiedades de un material.
- struct Transform

  Transformación de un objeto.

defines

- #define **WIDTH** 800
- #define **HEIGHT** 400
- #define **ASPECT_RATIO** (WIDTH / HEIGHT)
- #define **FOV_V** (M_PI / 3)
- #define **EXT_V** 100
- #define **NEAR** 0.1
- #define **FAR** 1000
- #define **KERNEL_RADIUS** 1
- #define **KERNEL_SIZE** 3
- #define **QUEUE_SIZE** 401
- #define **NUM_LIGHTS** 2

Funciones

- Payload vertex_shader (Vertex vertex, double ∗position)

  Genera la posición final de un vértice.
- void fragment_shader (Payload in, double ∗color)

  Genera el color final de un fragmento.
- bool **load_obj** (std::string path)

  Construye el buffer de vértices a partir de un archivo obj con caras formadas por cuadriláteros.

Variables

- QImage **image**

  La imagen que se muestra en la ventana.
- std::vector< Vertex > **vertices**

  Los vértices del cubo.
- Transform **cube_transform**

  La transformación del cubo.
- Material **cube_material**

  El material del cubo.
- double **camera_positions** [2][3]

  Las posiciones de las cámaras.
- double **camera_target** [3]

  El punto hacia donde ven las cámaras.
- double **camera_ref** [3]

  El vector de referencia hacia arriba de las cámaras.
- double **model_mat** [4][4]

  La matriz de modelo.
- double **view_mat** [4][4]

  La matriz de vista.
- double **projection_mat** [4][4]

  La matriz de proyección.
- double **normal_mat** [4][4]

  La matriz normal.
- double **mvp_mat** [4][4]

  El producto de las matrices de proyección, vista y modelo.
- void ∗ **mvp_chain** [3]

  Arreglo con las matrices de proyección, vista y modelo.
- bool **perspective_flag**

  Bandera que indica si la proyección actual es en perspectiva.
- Light **lights** [NUM_LIGHTS]

  Arreglo con las luces de la escena.
- double **global_ambient** [3]

  Componentes de la luz ambiental global de la escena.
- uint64_t **active_lights**

  Máscara de las luces encendidas.
- uint64_t **camera_id**

  Índice de la cámara seleccionada.
- uint64_t **light_id**

  Índice de la luz seleccionada.
- bool **filter**

Bandera que indica si se aplica el filtro de posprocesamiento.

- Kernel **kernel**

  Kernel usado en el filtro de posprocesamiento.

- uint32_t **queue** [QUEUE_SIZE]

  Cola usada para las operaciones de convolución.

- uint32_t **color_buffer** [HEIGHT][WIDTH]

  Buffer de color que forma la imagen mostrada en el lienzo.

- double **depth_buffer** [HEIGHT][WIDTH]

  Buffer usado en la prueba de profundidad.

5.1.1. Documentación de funciones

5.1.1.1. fragment_shader()

```
void fragment_shader (
            Payload in,
            double * color)
```

Genera el color final de un fragmento.

Parámetros

in	es la carga que se recibe
color	es donde se guarda el color final

5.1.1.2. vertex_shader()

```
Payload vertex_shader (
            Vertex vertex,
            double * position)
```

Genera la posición final de un vértice.

Parámetros

vertex	es el vértice
position	es donde se guarda la posición final

Devuelve

la carga que se envía a los fragmentos

5.2. app.hpp

Ir a la documentación de este archivo.

```
# pragma once

# include <QtWidgets>

# include "tiny_obj_loader.h"
# include "gmath.hpp"
# include "dip.hpp"

# define WIDTH 800
# define HEIGHT 400
# define ASPECT_RATIO (WIDTH / HEIGHT)
# define FOV_V (M_PI / 3)
# define EXT_V 100
# define NEAR 0.1
# define FAR 1000
# define KERNEL_RADIUS 1
# define KERNEL_SIZE 3
# define QUEUE_SIZE 401
# define NUM_LIGHTS 2

struct Vertex {
    double position [3];
    double normal [3];
    double color [3];
};

struct Payload {
    double view [3];
    double normal [3];
    double light [NUM_LIGHTS] [3];
    double color [3];
};

struct Light {
    double position [3];
    double ambient [3];
    double diffuse [3];
    double specular [3];
};

struct Material {
    double ambient [3];
    double diffuse [3];
    double specular [3];
    double shininess;
};

struct Transform {
    double position [3];
    double rotation [4];
    double scale [3];
};

extern QImage image;

extern std::vector <Vertex> vertices;

extern Transform cube_transform;

extern Material cube_material;

extern double camera_positions [2] [3];

extern double camera_target [3];

extern double camera_ref [3];

extern double model_mat [4] [4];

extern double view_mat [4] [4];

extern double projection_mat [4] [4];

extern double normal_mat [4] [4];

extern double mvp_mat [4] [4];

extern void * mvp_chain [3];

extern bool perspective_flag;
```

```
00102
00104 extern Light lights [NUM_LIGHTS];
00105
00107 extern double global_ambient [3];
00108
00110 extern uint64_t active_lights;
00111
00113 extern uint64_t camera_id;
00114
00116 extern uint64_t light_id;
00117
00119 extern bool filter;
00120
00122 extern Kernel kernel;
00123
00125 extern uint32_t queue [QUEUE_SIZE];
00126
00128 extern uint32_t color_buffer [HEIGHT] [WIDTH];
00129
00131 extern double depth_buffer [HEIGHT] [WIDTH];
00132
00137 Payload vertex_shader (Vertex vertex, double * position);
00138
00142 void fragment_shader (Payload in, double * color);
00143
00145 bool load_obj (std::string path);
```

5.3. Referencia del archivo include/dip.hpp

```
#include <cstdint>
#include <cmath>
#include <QColor>
```

Clases

- struct Coord

  Coordenadas de un pixel en una imagen.
- struct Rect

  Región rectangular de pixeles en una imagen.
- struct Extent

  Tamaño de una región rectangular en una imagen.
- struct Image

  Imagen con información del tamaño.
- struct Kernel

  Kernel simétrico para operaciones de convolución.

Funciones

- uint8_t to_color_component (double value)

  Convierte un valor flotante a un componente entero de color de 8 bits.
- uint32_t apply_kernel (Image image, Rect window, Kernel kernel, Coord center)

  Calcula la suma de los productos de cada elemento de `image` en `window` por el elemento correspondiente en `kernel`.
- Rect build_kernel_window (Rect rect, Coord center, uint64_t radius)

  Construye una ventana con centro en un pixel dado para aplicar un kernel.
- void convolution_filter_h (Image image, Rect rect, Kernel kernel, uint32_t *queue)
- void convolution_filter_v (Image image, Rect rect, Kernel kernel, uint32_t *queue)
- void convolution_filter (Image image, Rect rect, Kernel kernel, uint32_t *queue)

5.3.1. Documentación de funciones

5.3.1.1. apply_kernel()

```
uint32_t apply_kernel (
            Image image,
            Rect window,
            Kernel kernel,
            Coord center)
```

Calcula la suma de los productos de cada elemento de `image` en `window` por el elemento correspondiente en `kernel`.

Parámetros

image	es la imagen
window	es la ventana
kernel	es el kernel
center	son las coordenadas del centro de la ventana en la imagen

Devuelve

la suma de los productos de cada elemento en `window` por el elemento correspondiente en `kernel`

5.3.1.2. build_kernel_window()

```
Rect build_kernel_window (
            Rect rect,
            Coord center,
            uint64_t radius)
```

Construye una ventana con centro en un pixel dado para aplicar un kernel.

Parámetros

rect	es la región de la imagen donde se aplica la operación de convolución
center	son las coordenadas del centro de la ventana en la imagen
radius	es el radio del kernel

Devuelve

una ventana centrada en `center` para un kernel de radio `radius` que no se sale de la región `rect`

5.3.1.3. convolution_filter()

```
void convolution_filter (
            Image image,
            Rect rect,
            Kernel kernel,
            uint32_t * queue)
```

Aplica un filtro de convolución

Parámetros

image	es la imagen

rect	es la región de la imagen donde se aplica el filtro
kernel	es el kernel
queue	es una cola de capacidad al menos `kernel.radius * (min (image.extent.width, image.extent.height) + 1)`

5.3.1.4. convolution_filter_h()

```
void convolution_filter_h (
            Image image,
            Rect rect,
            Kernel kernel,
            uint32_t * queue)
```

Aplica un filtro de convolución en orden primero horizontal

Parámetros

image	es la imagen
rect	es la región de la imagen donde se aplica el filtro
kernel	es el kernel
queue	es una cola de capacidad al menos `kernel.radius * (image.extent.width + 1)`

5.3.1.5. convolution_filter_v()

```
void convolution_filter_v (
            Image image,
            Rect rect,
            Kernel kernel,
            uint32_t * queue)
```

Aplica un filtro de convolución en orden primero vertical

Parámetros

image	es la imagen
rect	es la región de la imagen donde se aplica el filtro
kernel	es el kernel
queue	es una cola de capacidad al menos `kernel.radius * (image.extent.height + 1)`

5.3.1.6. to_color_component()

```
uint8_t to_color_component (
            double value)
```

Convierte un valor flotante a un componente entero de color de 8 bits.

Parámetros

value	el valor flotante

Devuelve

el componente de color correspondiente

5.4. dip.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include <cstdint>
00006 # include <cmath>
00007
00008 # include <QColor>
00009
00011 struct Coord {
00012     uint64_t x, y;
00013 };
00014
00016 struct Rect {
00017     Coord start, end;
00018 };
00019
00021 struct Extent {
00022     uint64_t width, height;
00023 };
00024
00026 struct Image {
00027     Extent extent;
00028     void * data;
00029 };
00030
00032 struct Kernel {
00033     uint64_t radius;
00034     void * data;
00035 };
00036
00040 uint8_t to_color_component (double value);
00041
00048 uint32_t apply_kernel (Image image, Rect window, Kernel kernel, Coord center);
00049
00055 Rect build_kernel_window (Rect rect, Coord center, uint64_t radius);
00056
00062 void convolution_filter_h (Image image, Rect rect, Kernel kernel, uint32_t * queue);
00063
00069 void convolution_filter_v (Image image, Rect rect, Kernel kernel, uint32_t * queue);
00070
00076 void convolution_filter (Image image, Rect rect, Kernel kernel, uint32_t * queue);
```

5.5. Referencia del archivo include/gmath.hpp

```
#include <cstdint>
#include <cstring>
#include <cmath>
```

Funciones

- double clamp (double x, double a, double b)

  Limita un valor `x` *a un intervalo* `[a,b]`.
- double lerp (double t, double a, double b)

  Realiza una interpolación lineal para `t`.
- double dlerp (double t, double a, double b)

  Realiza una interpolación lineal para `t`.

- double blerp (double u, double v, double a, double b, double c)

  Realiza una interpolación lineal para `u` y `v`.
- double vec_norm (double ∗vec, uint64_t n)

  Calcula la norma de un vector.
- void vec_unit (double ∗vec_a, double ∗vec_b, uint64_t n)

  Construye el vector unitario en la dirección del vector `vec`.
- void vec_to_homogeneous (double ∗vec_a, double ∗vec_b, uint64_t n)

  Convierte un vector en coordenadas normales a coordenadas homogéneas.
- void vec_from_homogeneous (double ∗vec_a, double ∗vec_b, uint64_t n)

  Convierte un vector en coordenadas homogeneas a coordenadas normales.
- void vec_add (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Suma dos vectores.
- void vec_sub (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Resta dos vectores.
- void vec_emul (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Multiplica dos vectores elemento por elemento.
- void vec_scalar_mul (double ∗vec_a, double scalar, double ∗vec_b, uint64_t n)

  Multiplica un vector por un escalar.
- double vec_dot_prod (double ∗vec_a, double ∗vec_b, uint64_t n)

  Calcula el producto punto entre dos vectores.
- void vec3_cross_prod (double ∗vec_a, double ∗vec_b, double ∗vec_c)

  Calcula el producto cruz entre vectores de 3 dimensiones.
- void vec_reflect (double ∗incident, double ∗normal, double ∗reflected, uint64_t n)

  Construye el vector unitario reflejado.
- void mat_identity (void ∗mat, uint64_t n)

  Construye la matriz identidad.
- void mat_transpose (void ∗mat_a, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Construye la transpuesta de una matriz.
- void mat_add (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Suma dos matrices.
- void mat_sub (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Resta dos matrices.
- void mat_emul (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Multiplica dos matrices elemento por elemento.
- void mat_scalar_mul (void ∗mat_a, double scalar, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Multiplica una matriz por un escalar.
- void mat_vec_mul (void ∗mat, double ∗vec_a, double ∗vec_b, uint64_t n_1, uint64_t n_2)

  Multiplica una matriz por un vector columna.
- void mat_mul (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2, uint64_t n_3)

  Multiplica dos matrices.
- double mat_inverse (void ∗mat_a, void ∗mat_b, uint64_t n)

  Construye la inversa de una matriz. Se pierde el contenido de `mat_a`.
- void camera_look_at_mat4 (double ∗position, double ∗target, double ∗ref, void ∗mat)

  Construye la matriz de vista para una cámara en `position` viendo hacia `target` con vector de referencia hacia arriba `ref`.
- void orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void ∗mat)

  Construye la matriz de proyección ortogonal.
- void perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void ∗mat)

  Construye la matriz de proyección en perspectiva.
- void transform_euler_zyx_mat4 (double ∗position, double ∗rotation, double ∗scale, void ∗mat)

Construye la matriz de transformación a partir del desplazamiento, rotación y escalamiento. Se considera que la rotación está dada por ángulos de euler ZYX. El orden de la transformación es: escalamiento, rotación alrededor de Z, rotación alrededor de Y, rotación alrededor de X y traslación.

- void mat4_chain_mul (void ∗∗array, uint64_t n, void ∗mat)

  Multiplica una secuencia de matrices 4 4.
- void bound_points (void ∗mat, void ∗aabb, uint64_t n_1, uint64_t n_2)

  Calcula la caja delimitadora alineada con los ejes mínima para los puntos en las filas de la matriz `mat`*.*
- void linear_param (void ∗mat_a, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Construye una parametrización lineal para los puntos en las filas de la matriz `mat_a`*.*
- bool aabb_empty (void ∗aabb, uint64_t n)

  Determina si una caja delimitadora alineada con los ejes es vacía.
- void aabb_intersection (void ∗aabb_a, void ∗aabb_b, void ∗aabb_c, uint64_t n)

  Intersecta dos cajas delimitadoras alineadas con los ejes.
- void line_normal (double ∗point_a, double ∗point_b, double ∗normal)

  Calcula la normal de la recta que pasa por dos puntos en el plano.
- void plane_normal (double ∗point_a, double ∗point_b, double ∗point_c, double ∗normal)

  Calcula la normal del plano que pasa por tres puntos en el espacio.

5.5.1. Documentación de funciones

5.5.1.1. aabb_empty()

```
bool aabb_empty (
            void * aabb,
            uint64_t n)
```

Determina si una caja delimitadora alineada con los ejes es vacía.

Parámetros

aabb	es el caja
n	es la dimensión de `aabb`

Devuelve

`true` si y solo si la caja está vacía

5.5.1.2. aabb_intersection()

```
void aabb_intersection (
            void * aabb_a,
            void * aabb_b,
            void * aabb_c,
            uint64_t n)
```

Intersecta dos cajas delimitadoras alineadas con los ejes.

Parámetros

aabb↩ _a	es la primer caja

aabb↩ _b	es la segunda caja
aabb↩ _c	es donde se guarda el resultado
n	es la dimensión de `aabb_a` y `aabb_b`

5.5.1.3. blerp()

```
double blerp (
            double u,
            double v,
            double a,
            double b,
            double c)
```

Realiza una interpolación lineal para `u` y `v`.

Parámetros

u	es el primer valor de interpolación
v	es el segundo valor de interpolación
a	es el resultado si `u = 0` y `v = 0`
b	es el resultado si `u = 1` y `v = 0`
c	es el resultado si `u = 0` y `v = 1`

Devuelve

la interpolación lineal para `u` y `v`

5.5.1.4. bound_points()

```
void bound_points (
            void * mat,
            void * aabb,
            uint64_t n_1,
            uint64_t n_2)
```

Calcula la caja delimitadora alineada con los ejes mínima para los puntos en las filas de la matriz `mat`.

Parámetros

mat	es la matriz con los puntos
aabb	es donde se guarda el resultado
n_1	es el número de filas de `mat`

n_2	es el número de columnas de `mat`

5.5.1.5. camera_look_at_mat4()

```
void camera_look_at_mat4 (
            double * position,
            double * target,
            double * ref,
            void * mat)
```

Construye la matriz de vista para una cámara en `position` viendo hacia `target` con vector de referencia hacia arriba `ref`.

Parámetros

position	es la posición de la cámara
target	es la posición adonde apunta la cámara
ref	es el vector de referencia hacia arriba
mat	es donde se guarda el resultado

5.5.1.6. clamp()

```
double clamp (
            double x,
            double a,
            double b)
```

Limita un valor `x` a un intervalo `[a,b]`.

Parámetros

x	es el valor
a	es el límite inferior del intervalo
b	es el límite superior del intervalo

Devuelve

`x` si se encuentra en `[a,b]` o el límite más cercano a `x` si no está en el intervalo

5.5.1.7. dlerp()

```
double dlerp (
            double t,
            double a,
            double b)
```

Realiza una interpolación lineal para `t`.

Parámetros

t	es el valor de interpolación

a	es el resultado si `t = -1`
b	es el resultado si `t = 1`

Devuelve

la interpolación lineal para `t`

5.5.1.8. lerp()

```
double lerp (
            double t,
            double a,
            double b)
```

Realiza una interpolación lineal para `t`.

Parámetros

t	es el valor de interpolación
a	es el resultado si `t = 0`
b	es el resultado si `t = 1`

Devuelve

la interpolación lineal para `t`

5.5.1.9. line_normal()

```
void line_normal (
            double * point_a,
            double * point_b,
            double * normal)
```

Calcula la normal de la recta que pasa por dos puntos en el plano.

Parámetros

point↩_a	es el primer punto
point↩_b	es el segundo punto
normal	es donde se guarda el resultado

5.5.1.10. linear_param()

```
void linear_param (
            void * mat_a,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Construye una parametrización lineal para los puntos en las filas de la matriz `mat_a`.

Parámetros

mat↩ _a	es la matriz con los puntos

mat↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.11. mat4_chain_mul()

```
void mat4_chain_mul (
            void ** array,
            uint64_t n,
            void * mat)
```

Multiplica una secuencia de matrices 4 4.

Parámetros

array	es el arreglo con las matrices
n	es el tamaño del arreglo
mat	es donde se guarda el resultado

5.5.1.12. mat_add()

```
void mat_add (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2)
```

Suma dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.13. mat_emul()

```
void mat_emul (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2)
```

Multiplica dos matrices elemento por elemento.

Parámetros

mat↩_a	es la primer matriz
mat↩_b	es la segunda matriz
mat↩_c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.14. mat_identity()

```
void mat_identity (
            void * mat,
            uint64_t n)
```

Construye la matriz identidad.

Parámetros

mat	es donde se guarda el resultado
n	es el número de filas y columnas de la matriz

5.5.1.15. mat_inverse()

```
double mat_inverse (
            void * mat_a,
            void * mat_b,
            uint64_t n)
```

Construye la inversa de una matriz. Se pierde el contenido de `mat_a`.

Parámetros

mat↩_a	es la matriz

mat↩ _b	es donde se guarda el resultado
n	es el número de filas y columnas de `mat_a`

Devuelve

el determinante de `mat_a`

5.5.1.16. mat_mul()

```
void mat_mul (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2,
            uint64_t n_3)
```

Multiplica dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a` y el número de filas de `mat_b`
n_3	es el número de columnas de `mat_b`

5.5.1.17. mat_scalar_mul()

```
void mat_scalar_mul (
            void * mat_a,
            double scalar,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Multiplica una matriz por un escalar.

Parámetros

mat↩ _a	es la matriz

scalar	es el escalar
mat↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.18. mat_sub()

```
void mat_sub (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2)
```

Resta dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.19. mat_transpose()

```
void mat_transpose (
            void * mat_a,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Construye la transpuesta de una matriz.

Parámetros

mat↩ _a	es la matriz
mat↩ _b	es donde se guarda el resultado

n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.20. mat_vec_mul()

```
void mat_vec_mul (
            void * mat,
            double * vec_a,
            double * vec_b,
            uint64_t n_1,
            uint64_t n_2)
```

Multiplica una matriz por un vector columna.

Parámetros

mat	es la matriz
vec↩ _a	es el vector columna
vec↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat`
n_2	es el número de columnas de `mat` y `vec_a`

5.5.1.21. orthogonal_proj_mat4()

```
void orthogonal_proj_mat4 (
            double aspect_ratio,
            double ext_v,
            double near,
            double far,
            void * mat)
```

Construye la matriz de proyección ortogonal.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión
ext_v	es la extensión vertical del volumen de visión
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.22. perspective_proj_mat4()

```
void perspective_proj_mat4 (
            double aspect_ratio,
            double fov_v,
            double near,
            double far,
            void * mat)
```

Construye la matriz de proyección en perspectiva.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión
fov_v	es el campo de visión vertical
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.23. plane_normal()

```
void plane_normal (
            double * point_a,
            double * point_b,
            double * point_c,
            double * normal)
```

Calcula la normal del plano que pasa por tres puntos en el espacio.

Parámetros

point↩ _a	es el primer punto
point↩ _b	es el segundo punto
point↩ _c	es el tercer punto
normal	es donde se guarda el resultado

5.5.1.24. transform_euler_zyx_mat4()

```
void transform_euler_zyx_mat4 (
            double * position,
            double * rotation,
            double * scale,
            void * mat)
```

Construye la matriz de transformación a partir del desplazamiento, rotación y escalamiento. Se considera que la rotación está dada por ángulos de euler ZYX. El orden de la transformación es: escalamiento, rotación alrededor de Z, rotación alrededor de Y, rotación alrededor de X y traslación.

Parámetros

position	es el desplazamiento

rotation	es la rotación en ángulos de euler ZYX
scale	es el escalamiento
mat	es donde se guarda el resultado

5.5.1.25. vec3_cross_prod()

```
void vec3_cross_prod (
            double * vec_a,
            double * vec_b,
            double * vec_c)
```

Calcula el producto cruz entre vectores de 3 dimensiones.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado

5.5.1.26. vec_add()

```
void vec_add (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Suma dos vectores.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.27. vec_dot_prod()

```
double vec_dot_prod (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Calcula el producto punto entre dos vectores.

Parámetros

vec_a	es el primer vector

vec↩ _b	es el segundo vector
n	es la dimensión `vec_a` y `vec_b`

Devuelve

el producto punto de ambos vectores

5.5.1.28. vec_emul()

```
void vec_emul (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Multiplica dos vectores elemento por elemento.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.29. vec_from_homogeneous()

```
void vec_from_homogeneous (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Convierte un vector en coordenadas homogeneas a coordenadas normales.

Parámetros

vec↩ _a	es el vector en coordenadas homogéneas
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.30. vec_norm()

```
double vec_norm (
            double * vec,
            uint64_t n)
```

Calcula la norma de un vector.

Parámetros

vec	es el vector
n	es la dimensión de `vec`

Devuelve

la norma de `vec`

5.5.1.31. vec_reflect()

```
void vec_reflect (
            double * incident,
            double * normal,
            double * reflected,
            uint64_t n)
```

Construye el vector unitario reflejado.

Parámetros

incident	es el vector unitario incidente
normal	es el vector unitario de la normal
reflected	es donde se guarda el resultado
n	es la dimensión de `incident` y `normal`

5.5.1.32. vec_scalar_mul()

```
void vec_scalar_mul (
            double * vec_a,
            double scalar,
            double * vec_b,
            uint64_t n)
```

Multiplica un vector por un escalar.

Parámetros

vec↩ _a	es el vector

scalar	es el escalar
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.33. vec_sub()

```
void vec_sub (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Resta dos vectores.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.34. vec_to_homogeneous()

```
void vec_to_homogeneous (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Convierte un vector en coordenadas normales a coordenadas homogéneas.

Parámetros

vec↩ _a	es el vector en coordenadas normales
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.35. vec_unit()

```
void vec_unit (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Construye el vector unitario en la dirección del vector `vec`.

Parámetros

vec↩ _a	es el vector

vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.6. gmath.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include <cstdint>
00006 # include <cstring>
00007 # include <cmath>
00008
00014 double clamp (double x, double a, double b);
00015
00021 double lerp (double t, double a, double b);
00022
00028 double dlerp (double t, double a, double b);
00029
00037 double blerp (double u, double v, double a, double b, double c);
00038
00043 double vec_norm (double * vec, uint64_t n);
00044
00049 void vec_unit (double * vec_a, double * vec_b, uint64_t n);
00050
00055 void vec_to_homogeneous (double * vec_a, double * vec_b, uint64_t n);
00056
00061 void vec_from_homogeneous (double * vec_a, double * vec_b, uint64_t n);
00062
00068 void vec_add (double * vec_a, double * vec_b, double * vec_c, uint64_t n);
00069
00075 void vec_sub (double * vec_a, double * vec_b, double * vec_c, uint64_t n);
00076
00082 void vec_emul (double * vec_a, double * vec_b, double * vec_c, uint64_t n);
00083
00089 void vec_scalar_mul (double * vec_a, double scalar, double * vec_b, uint64_t n);
00090
00096 double vec_dot_prod (double * vec_a, double * vec_b, uint64_t n);
00097
00102 void vec3_cross_prod (double * vec_a, double * vec_b, double * vec_c);
00103
00109 void vec_reflect (double * incident, double * normal, double * reflected, uint64_t n);
00110
00114 void mat_identity (void * mat, uint64_t n);
00115
00121 void mat_transpose (void * mat_a, void * mat_b, uint64_t n_1, uint64_t n_2);
00122
00129 void mat_add (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);
00130
00137 void mat_sub (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);
00138
00145 void mat_emul (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);
00146
00153 void mat_scalar_mul (void * mat_a, double scalar, void * mat_b, uint64_t n_1, uint64_t n_2);
00154
00161 void mat_vec_mul (void * mat, double * vec_a, double * vec_b, uint64_t n_1, uint64_t n_2);
00162
00170 void mat_mul (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2, uint64_t n_3);
00171
00177 double mat_inverse (void * mat_a, void * mat_b, uint64_t n);
00178
00184 void camera_look_at_mat4 (double * position, double * target, double * ref, void * mat);
00185
00192 void orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void * mat);
00193
00200 void perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void * mat);
00201
00209 void transform_euler_zyx_mat4 (double * position, double * rotation, double * scale, void * mat);
00210
00215 void mat4_chain_mul (void * * array, uint64_t n, void * mat);
00216
00222 void bound_points (void * mat, void * aabb, uint64_t n_1, uint64_t n_2);
00223
00229 void linear_param (void * mat_a, void * mat_b, uint64_t n_1, uint64_t n_2);
00230
00235 bool aabb_empty (void * aabb, uint64_t n);
```

```
00236
00242 void aabb_intersection (void * aabb_a, void * aabb_b, void * aabb_c, uint64_t n);
00243
00248 void line_normal (double * point_a, double * point_b, double * normal);
00249
00255 void plane_normal (double * point_a, double * point_b, double * point_c, double * normal);
```

5.7. Referencia del archivo include/gui.hpp

```
#include "pipeline.hpp"
```

Clases

- struct Canvas

  Lienzo donde se muestra la imagen en la ventana.
- struct ViewDock

  Puerto para el control de la proyección.
- struct TransformDock

  Puerto para el control de la transformación.
- struct LightsDock

  Puerto para el control de las luces.
- struct MaterialDock

  Puerto para el control del material.
- struct PostProcessingDock
- struct Window

  Ventana principal de la aplicación.

5.8. gui.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include "pipeline.hpp"
00006
00008 struct Canvas : public QWidget {
00009
00012     void paintEvent(QPaintEvent * event);
00013
00014 };
00015
00017 struct ViewDock : public QDockWidget {
00018
00019     QWidget widget;
00020     QVBoxLayout layout;
00021     QGroupBox cam_group;
00022     QVBoxLayout cam_group_layout;
00023     QRadioButton cam_group_buttons [2];
00024     QGroupBox proj_group;
00025     QVBoxLayout proj_group_layout;
00026     QRadioButton proj_group_buttons [2];
00027
00029     ViewDock ();
00030
00032     void syncValues ();
00033
00034 };
00035
00037 struct TransformDock : public QDockWidget {
00038
```

```
    QWidget widget;
    QGridLayout layout;
    QLabel row_labels [3];
    QLabel col_labels [3];
    QDoubleSpinBox pos_boxes [3];
    QDoubleSpinBox rot_boxes [3];
    QDoubleSpinBox scale_boxes [3];

    TransformDock ();

    void syncValues ();

};

struct LightsDock : public QDockWidget {

    QWidget widget;
    QGridLayout layout;
    QComboBox dropdown;
    QCheckBox checkbox;
    QLabel row_labels [3];
    QLabel col_labels [3];
    QDoubleSpinBox amb_boxes [3];
    QDoubleSpinBox diff_boxes [3];
    QDoubleSpinBox spec_boxes [3];

    LightsDock ();

    void syncValues ();

};

struct MaterialDock : public QDockWidget {

    QWidget widget;
    QGridLayout layout;
    QLabel row_labels [3];
    QLabel col_labels [3];
    QDoubleSpinBox amb_boxes [3];
    QDoubleSpinBox diff_boxes [3];
    QDoubleSpinBox spec_boxes [3];
    QLabel sh_label;
    QDoubleSpinBox sh_box;

    MaterialDock ();

    void syncValues ();

};

struct PostProcessingDock : public QDockWidget {

    QWidget widget;
    QGridLayout layout;
    QCheckBox checkbox;
    QLabel label;
    QDoubleSpinBox boxes [3] [3];

    PostProcessingDock ();

    void syncValues ();

};

struct Window : public QMainWindow {

    Canvas canvas;
    ViewDock view_dock;
    TransformDock transform_dock;
    LightsDock lights_dock;
    MaterialDock material_dock;
    PostProcessingDock post_processing_dock;

    Window ();

    void draw ();

    void sync_all ();

    void connect_all ();

    void slot_pos_x (double value);

    void slot_pos_y (double value);

    void slot_pos_z (double value);

```

```
    void slot_rot_x (double value);

    void slot_rot_y (double value);

    void slot_rot_z (double value);

    void slot_scale_x (double value);

    void slot_scale_y (double value);

    void slot_scale_z (double value);

    void slot_proj (bool checked);

    void slot_cam (bool checked);

    void slot_light_id (int index);

    void slot_light_state (Qt::CheckState state);

    void slot_light_amb_r (double value);

    void slot_light_amb_g (double value);

    void slot_light_amb_b (double value);

    void slot_light_diff_r (double value);

    void slot_light_diff_g (double value);

    void slot_light_diff_b (double value);

    void slot_light_spec_r (double value);

    void slot_light_spec_g (double value);

    void slot_light_spec_b (double value);

    void slot_material_amb_r (double value);

    void slot_material_amb_g (double value);

    void slot_material_amb_b (double value);

    void slot_material_diff_r (double value);

    void slot_material_diff_g (double value);

    void slot_material_diff_b (double value);

    void slot_material_spec_r (double value);

    void slot_material_spec_g (double value);

    void slot_material_spec_b (double value);

    void slot_material_sh (double value);

    void slot_post_processing_filter (Qt::CheckState state);

    void slot_post_processing_kernel_value (double value);

};
```

5.9. Referencia del archivo include/pipeline.hpp

```
#include "app.hpp"
```

Clases

- struct Fragment

  Fragmento generado por el rasterizador.
- struct Pipeline

  Línea de renderizado por rasterización.

Variables

- Pipeline **pipeline**

  Línea de renderizado de la aplicación.

5.10. pipeline.hpp

Ir a la documentación de este archivo.

```


# pragma once

# include "app.hpp"

struct Fragment {
    uint64_t x, y;
    double u, v, depth;
    double color [4];
};

struct Pipeline {

    uint64_t num_primitives, width, height, primitive_id, vertex_id;
    bool front_face_clockwise;
    Fragment fragment;
    Payload out [3];
    Payload in;
    uint64_t viewport [2] [2];
    uint64_t raster_rect [2] [2];
    double positions_3d [3] [3];
    double boundary_3d [2] [3];
    double positions_2d [3] [2];
    double boundary_2d [2] [2];
    void * color_buffer;
    void * depth_buffer;
    void * vertex_buffer;
    Payload (* vertex_shader) (Vertex, double *);
    void (* fragment_shader) (Payload, double *);

    void clear_buffers ();

    void draw ();

    bool clipping_test ();

    void to_plane ();

    void to_viewport ();

    void rasterize ();

    bool depth_test ();

    void build_payload ();

    void blend_fragment ();

};

extern Pipeline pipeline;
```